

DocuSense – RAG-Only Document Analysis Project

This document contains the full source code and design layout of the DocuSense project, an offline intelligent document analysis tool that uses a Retrieval-Augmented Generation (RAG) pipeline. It does not use Pydantic models and provides modules for PDF/DOCX ingestion, chunking, BM25 retrieval, TextRank summarization, spaCy-based entity recognition, compliance scanning, and FastAPI endpoints.

Project Layout

```
docusense/
├── app.py
├── requirements.txt
├── config.yaml
├── storage/
│   ├── corpus/
│   ├── chunks/
│   ├── faiss/
│   └── logs/
├── rag/
│   ├── pipeline.py
│   ├── retrievers.py
│   ├── chunkers.py
│   ├── rankers.py
│   ├── summarizer.py
│   ├── ner.py
│   ├── compliance.py
│   ├── index.py
│   └── utils.py
```

requirements.txt

```
fastapi
uvicorn
python-multipart
pdfminer.six
python-docx
rank-bm25
faiss-cpu
numpy
scikit-learn
spacy
networkx
nltk
pyyaml
```

app.py

```
from fastapi import FastAPI, UploadFile, File, Request
from rag.pipeline import RagPipeline
import os, yaml

app = FastAPI(title="DocuSense (RAG-only, no Pydantic)")

with open("config.yaml", "r") as f:
    CFG = yaml.safe_load(f)
PIPE = RagPipeline(CFG)

@app.post("/ingest")
async def ingest(file: UploadFile = File(...)):
    os.makedirs("storage/corpus", exist_ok=True)
    path = os.path.join("storage/corpus", file.filename)
    with open(path, "wb") as f:
```

```

        f.write(await file.read())
    doc_id = PIPE.ingest(path)
    return {"ok": True, "doc_id": doc_id}

@app.post("/query")
async def query(request: Request):
    payload = await request.json()
    q = payload.get("query", "")
    answer, evidence = PIPE.answer(q)
    return {"answer": answer, "evidence": evidence}

```

pipeline.py

```

class RagPipeline:
    def __init__(self, cfg):
        self.cfg = cfg
        self.idx = IndexManager(cfg)

    def ingest(self, path:str)->str:
        chunks = parse_and_chunk(path, self.cfg["chunking"])
        doc_id = uid(path)
        self.idx.add(doc_id, chunks)
        return doc_id

    def answer(self, query:str, top_k:int=6):
        hits = self.idx.retrieve(query, top_k=top_k)
        context = "\n".join(h["text"] for h in hits)
        answer = textrank_summary(context, max_sentences=3)
        return answer, hits

```

Other Modules

- chunkers.py: PDF/DOCX parsing and text chunking
- rankers.py: BM25 retrieval
- summarizer.py: TextRank summarization
- ner.py: spaCy NER extraction
- compliance.py: Regex-based policy scan
- utils.py: Helpers for uid, audit logging, JSON save/load